

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi - 590018, Karnataka

A Project Report on

**"SightLite: An Accessible AI-Powered Browser
Automation Agent"**

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Name	USN
Keertana	4AL23CS065
Arpita Gavi	4AL23CS023



ALVA'S
Education Foundation®

ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi)

Department of Computer Science and Engineering

Mijar, Moodbidri - 574225, Karnataka

ABSTRACT

The modern web is rich in dynamic content, single page applications and complex layouts. For users with visual impairments, and for anyone who wants to save effort on repetitive browsing work, interacting with such pages through a mouse and screen is slow and error prone. This project presents SightLite, an accessible, AI-powered Chrome extension that works as an intelligent browser automation agent. The user types a goal in natural language, such as "click login" or "search for cats then scroll down", and SightLite carries it out on the active web page.

SightLite is built on Chrome Manifest V3. A service worker orchestrates the agent through the states observe, perceive, plan, act and complete. A content script performs the actual clicks, typing and scrolling, and a side panel gives live feedback. To perceive the page, SightLite uses a lightweight ONNX object detection model (uidet-nano-int8) executed in an offscreen document, with automatic fallback to DOM scanning. Natural language goals are translated into structured action steps using Google Gemini 1.5 Flash, and a multi-level scoring algorithm matches the words of the user to the correct element on the page. Safety features such as confirmation for destructive actions, step limits and a timeout keep the agent under control. An optional FastAPI and PostgreSQL backend can log sessions for analytics.

Testing on real pages, including Google Search results, shows that the extension can scan hundreds of elements in a fraction of a second, highlight the target and execute the requested action, while showing every step in a readable timeline.

Keywords: Chrome extension, browser automation, accessibility, ONNX, computer vision, natural language processing, Gemini.

TABLE OF CONTENTS

Certificate	i
Acknowledgement.....	ii
Abstract.....	iii
Chapter 1: Introduction	1
1.1 Overview	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope of the Project	2
Chapter 2: Literature Survey and Proposed System	3
2.1 Existing Approaches	3
2.2 Limitations of Existing Approaches.....	3
2.3 Proposed System.....	3
Chapter 3: System Requirements	4
3.1 Hardware Requirements	4
3.2 Software Requirements	4
3.3 Functional Requirements	4
3.4 Non-Functional Requirements	4
Chapter 4: System Design	6
4.1 System Architecture.....	6
4.2 Agent Lifecycle.....	6
4.3 Technology Stack.....	7
Chapter 5: Implementation.....	8
5.1 Chrome Extension Modules	8
5.2 Vision-Based Perception	8
5.3 Natural Language Understanding	8
5.4 Element Matching Algorithm	9
5.5 Action Execution System	9
5.6 Safety Features.....	10
5.7 AI Explain and Answer	10
5.8 Optional Backend.....	10
Chapter 6: Results and Discussion	11
6.1 Dashboard.....	11
6.2 Agent Running on a Web Page.....	12
6.3 Discussion.....	12
Chapter 7: Testing.....	13
7.1 Test Cases.....	13
Chapter 8: Limitations and Future Enhancements	14
8.1 Limitations.....	14
8.2 Future Enhancements	14
Chapter 9: Conclusion.....	15
References.....	16

CHAPTER 1

INTRODUCTION

1.1 Overview

The web browser is the main gateway to information, education, banking and government services. Yet a large part of the modern web is designed for sighted users who can scan a page visually and use a mouse. Pages are built with frameworks such as React, Vue and Angular, content loads dynamically, and many controls are drawn inside shadow DOM, iframes or canvas. Such pages are difficult to navigate for users with visual impairments and time-consuming even for other users who perform the same steps again and again.

SightLite is an accessible, AI-powered Chrome extension that acts as an intelligent browser automation agent. The user gives an instruction in plain language, and the agent looks at the page, decides which element the user means, and performs the action. The name reflects the goal of a lightweight system that gives "sight" to a browser task.

1.2 Problem Statement

Existing browser automation tools such as Selenium and Playwright need programming knowledge, and traditional screen readers depend entirely on how well a website has been coded for accessibility. There is a need for a simple tool that accepts natural language commands, understands what is on the screen even when the DOM is complicated, and performs actions safely on behalf of the user.

1.3 Objectives

- To design a Chrome Manifest V3 extension that accepts goals written in natural language.
- To detect interactive user interface elements using a lightweight on-device vision model, with DOM scanning as a fallback.
- To match the words of the user to the right element on the page with a scoring algorithm.
- To execute click, type, scroll and highlight actions reliably on modern web applications.
- To provide AI-generated explanations and answers about selected text or an entire page.

- To protect the user by asking for confirmation before destructive actions and by limiting the agent's steps and runtime.
- To show every step of the agent in a clear, high-contrast interface suitable for accessibility.

1.4 Scope of the Project

SightLite works on the active tab of Google Chrome and Chromium-based browsers such as Edge. It supports the actions click, type, scroll, explain, answer and find. It cannot operate on internal browser pages such as `chrome://` URLs because of browser security restrictions. The backend for session logging is optional and is not required for the main functionality.

CHAPTER 2

LITERATURE SURVEY AND PROPOSED SYSTEM

2.1 Existing Approaches

- **Screen readers (NVDA, JAWS, TalkBack):** read out the content of a page and depend on correct semantic markup and ARIA labels.
- **Browser automation frameworks (Selenium, Playwright, Puppeteer):** automate tasks through scripts written by developers using selectors and are not meant for end users.
- **Voice assistants and browser voice control:** allow commands by speech but offer limited understanding of arbitrary web page content.
- **Cloud-based AI browser agents:** understand goals in natural language, but usually send screen content to remote servers, which raises privacy and cost concerns.

2.2 Limitations of Existing Approaches

- Scripts break when the page layout or class names change.
- DOM-only tools cannot see content rendered in canvas, shadow DOM or deeply nested iframes.
- Users need programming knowledge to write automation.
- Cloud agents may expose private screen data and often need subscriptions.

2.3 Proposed System

SightLite combines three ideas: (i) natural language understanding to convert a goal into structured steps, (ii) perception using a small vision model that runs in the browser, so the page is understood as a user sees it, and (iii) a robust action layer with several fallback strategies. Hence the system does not depend on a fixed selector or on the perfect accessibility of a site, and screenshots for perception are processed on the device.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or above
RAM	4 GB minimum (8 GB recommended)
Storage	200 MB free space for the extension and models
Display	1366 x 768 or higher
Internet	Required for Gemini API based features

3.2 Software Requirements

Component	Requirement
Operating system	Windows 10/11, Linux or macOS
Browser	Google Chrome or Microsoft Edge (Manifest V3 support)
Languages	JavaScript (ES6 modules), HTML5, CSS3, Python 3 (optional backend)
Libraries	ONNX Runtime Web (WASM / WebGPU), FastAPI (optional)
AI services	Google Gemini 1.5 Flash API
Database (optional)	PostgreSQL with Docker
Editor	Visual Studio Code

3.3 Functional Requirements

- Accept a goal typed by the user in the side panel.
- Split the goal into one or more executable steps.
- Scan the current page and list its interactive elements.
- Highlight the chosen element before acting on it.
- Perform click, type and scroll actions and report success or failure.
- Allow the user to pause or stop the agent at any time.
- Explain or answer questions about selected text or the whole page.

3.4 Non-Functional Requirements

- **Accessibility:** high contrast, clear status labels, keyboard navigation and ARIA labels.
- **Performance:** perception should complete in a small fraction of a second for typical pages.
- **Privacy:** screenshots are processed on the device by the ONNX model.
- **Reliability:** multiple fallbacks for perception and action.
- **Safety:** confirmation for risky actions, and limits on steps and time.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system follows the standard Manifest V3 structure. The side panel is the interface, the service worker is the brain of the agent, the content script works inside the web page, and an offscreen document runs the vision model. Heavy computation such as WebAssembly inference cannot run comfortably inside a service worker, so it is moved to the offscreen document, which has a DOM context. Figure 4.1 shows how these components communicate.

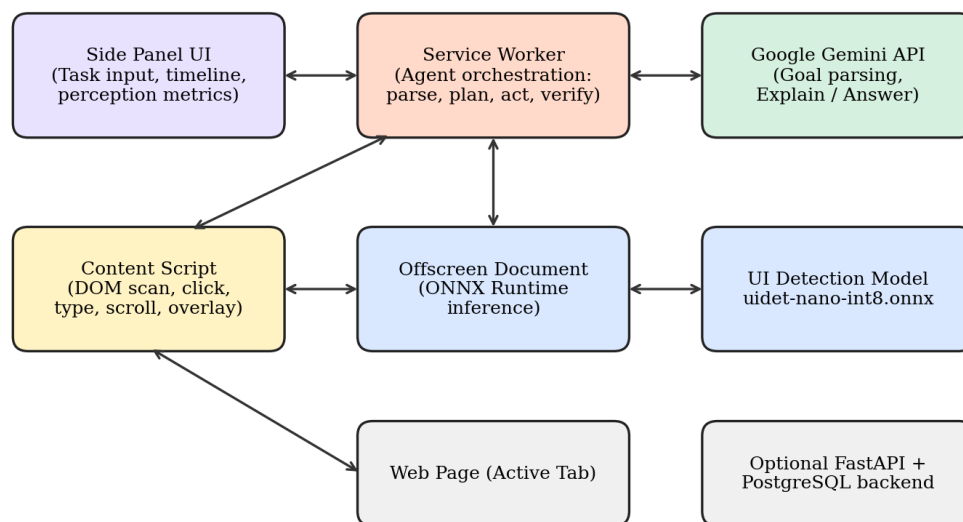


Figure 4.1: System architecture of SightLite

4.2 Agent Lifecycle

The service worker moves the agent through a sequence of states. Each state is shown in the side panel so that the user always knows what the agent is doing.

State	Description
IDLE	Ready to accept a command.
OBSERVING	Reading and parsing the goal of the user.
PERCEIVING	Scanning the page using the vision model or DOM.
PLANNING	Finding the target element for the current step.
ACTING	Executing the action on the page.
WAITING	Short delay after the action so the page can update.
COMPLETED	All steps finished successfully.

State	Description
ERROR	An action failed and the agent has stopped.

The overall flow is: User Input, Parse Goal, Perceive Page, Plan Action, Execute and Verify. The loop repeats for every step of the goal.

4.3 Technology Stack

Layer	Technology
Extension platform	Chrome Extension APIs, Manifest V3, Service Worker, Side Panel
User interface	HTML, CSS and vanilla JavaScript
Vision	ONNX Runtime with WASM and WebGPU; model uidet-nano-int8.onnx
Language understanding	Google Gemini 1.5 Flash
Backend (optional)	FastAPI, PostgreSQL, Docker

CHAPTER 5

IMPLEMENTATION

5.1 Chrome Extension Modules

Service worker (background/service-worker.js). It parses goals, manages the agent lifecycle, routes messages between the side panel, content scripts and the offscreen document, calls the Gemini API and executes page actions. Its main functions are `run(goal, tabId)`, `perceive(tabId)`, `pick(elements, keyword)`, `doAct(tabId, step, target)` and `parseGoalWithAI(goal)`.

Side panel (sidepanel/). It contains the task box, the Run, Pause, Stop and Clear buttons, a live activity timeline, perception metrics (mode, inference time and number of elements) and a status indicator.

Content script (content/content.js). It runs in the web page, scans the DOM, draws the highlight overlay and performs click, type, scroll and highlight actions on request from the service worker.

Offscreen document (offscreen/). It receives a screenshot from the service worker, runs the ONNX model and returns bounding boxes, labels and confidence scores.

5.2 Vision-Based Perception

The model `uidet-nano-int8.onnx` is a small object detector (about 5 MB) that finds buttons, input fields, links, select boxes, icons and images. The screenshot is resized to 640 x 640 pixels, and detections with confidence above 0.40 are kept. Because it looks at the rendered page, it can find elements which normal DOM queries miss, such as those in shadow DOM or canvas.

Perception follows a fallback chain so that the agent still works when a method fails:

- Vision detection using ONNX.
- DOM scanning using `querySelector`.
- Content script fallback.
- Report an error to the user.

In Figure 6.2 the extension used the DOM fallback mode and found 315 elements on a Google results page in under a millisecond.

5.3 Natural Language Understanding

The goal is sent to Google Gemini 1.5 Flash, which returns structured actions. Complex multi-step goals and ambiguous phrases are supported. Examples:

User goal	Structured output
search for cats	type into "search" the value "cats"
click login then type admin@example.com in email	click "login"; type "admin@example.com" into "email"
explain this article	explain "this"

The supported actions are click, type, scroll, explain, answer and find (highlight without clicking).

5.4 Element Matching Algorithm

After perception, the function pick() assigns a score to each element for the target words given by the user. The element with the highest score is selected.

Match type	Score	Example
Exact match	1.00	"login" matches the button "Login"
Word boundary match	0.98	"search" matches "Search Products"
Starts with	0.94	"submit" matches "Submit Form"
Contains word	0.88	"email" matches "Enter your email"
Multi-token	0.75	"sign in button" matches an element with both words
Fuzzy	0.60	Short partial matches

Hidden elements, elements outside the viewport, elements smaller than 1 pixel and noise words such as "button", "field" and "input" are ignored. Exact text, ARIA labels, placeholder text and input types are given priority, and elements that appear earlier in the DOM are preferred when scores are equal.

5.5 Action Execution System

Modern frameworks listen to specific events, and some elements are wrapped inside containers, so the agent tries several strategies in order.

Clicking

- Click through the content script.

- Direct click through `executeScript`.
- Find an ancestor `<a href>` and navigate directly.
- Find a container that holds the text and click its first link.
- Dispatch `mousedown`, `mouseup` and `click` events.

Typing

- Focus and click the element.
- Use `document.execCommand("insertText")`.
- Set the value directly.
- Trigger `input`, `change`, `keydown` and `keyup` events so React and Vue detect the change.

For single page applications such as a paper search portal, the agent waits for dynamic content, scans all text nodes (not only buttons), finds the parent card of the text and navigates to the link inside it.

5.6 Safety Features

- Confirmation is requested before delete, remove, payment, purchase, checkout, logout and sign out operations.
- Typing into password fields is not allowed.
- A maximum of 15 steps per session and a 90 second timeout.
- A maximum of 2 retries for perception.
- The target element is highlighted for 300 ms before the action, so the user can see what will happen.

5.7 AI Explain and Answer

The user can select text on a page and choose "Explain with SightLite" or "Answer with SightLite" from the right-click menu. The text is sent to Gemini and a short explanation of two to three sentences is displayed in the timeline. For commands such as "explain this article", the main content is extracted from `<article>`, `<main>`, `[role="main"]`, common article classes or long paragraphs, while navigation, headers and footers are filtered out.

5.8 Optional Backend

A FastAPI and PostgreSQL backend can store sessions for analytics. It exposes the endpoints `POST /sessions`, `POST /sessions/{id}/frames`, `POST /sessions/{id}/actions` and

POST /sessions/{id}/end. The data recorded includes the goal, page URL, number of detected elements, inference latency, attempted actions and the success or failure rate. The extension works fully without this backend.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Dashboard

The SightLite dashboard gives an overview of the system: session count, frames processed, average latency and success rate, along with the system status and recent activity. It also offers quick actions for live perception, screen analysis, task automation and session history. The interface supports a dark mode and shows an "On-Device Mode" indicator, which reflects the privacy focus of the project.

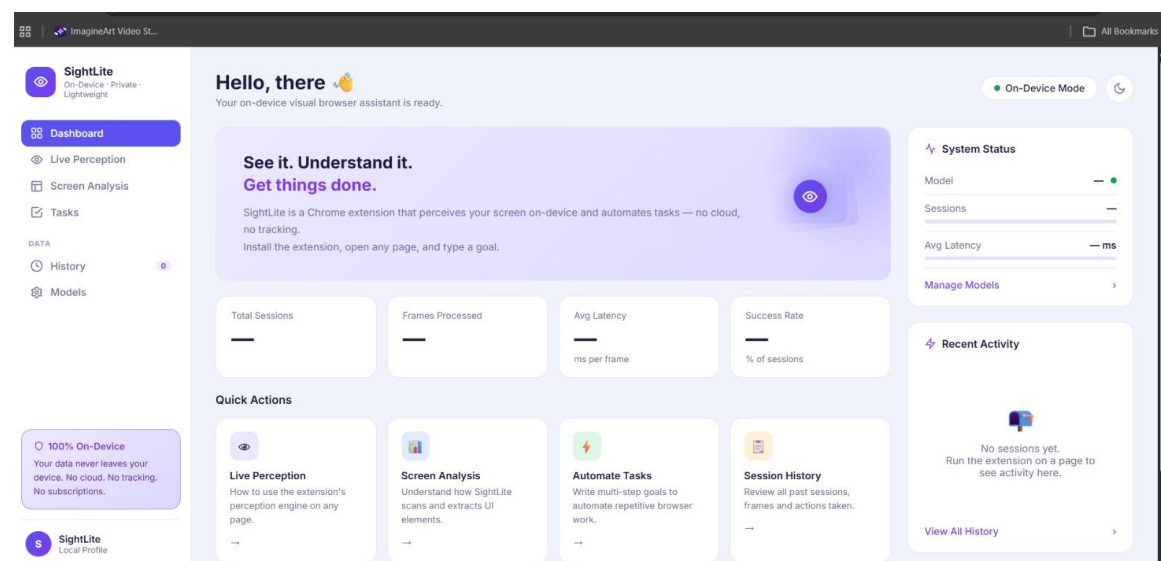


Figure 6.1(a): SightLite dashboard (initial version)

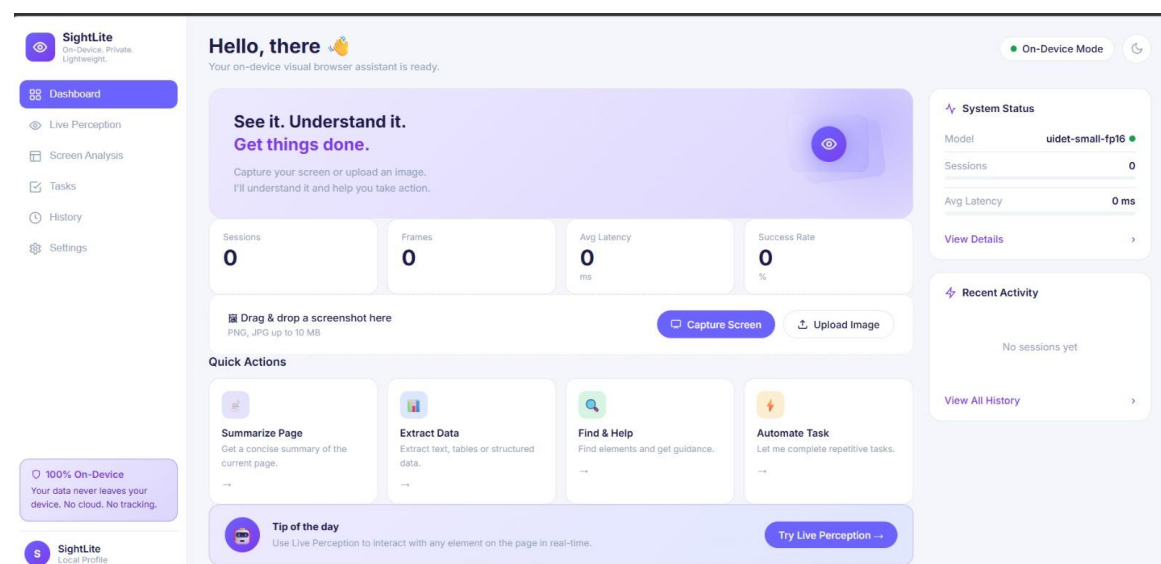


Figure 6.1(b): SightLite dashboard (updated version with capture and upload options)

The updated version adds a drag and drop area, the Capture Screen and Upload Image buttons, the active model name (uidet-small-fp16) and quick actions such as Summarize Page, Extract Data, Find & Help and Automate Task.

6.2 Agent Running on a Web Page

Figure 6.2 shows the extension working in the side panel on a Google search results page for the query "weather mangalore". The goal typed was "click AI Mode". The timeline shows the parsed goal with one step, the scanning of the page, waiting for page content, the DOM scan that found 315 elements, and the search for the target "AI Mode". The perception panel reports the mode as DOM Fallback with inference time below 1 ms, and the status is WAITING.

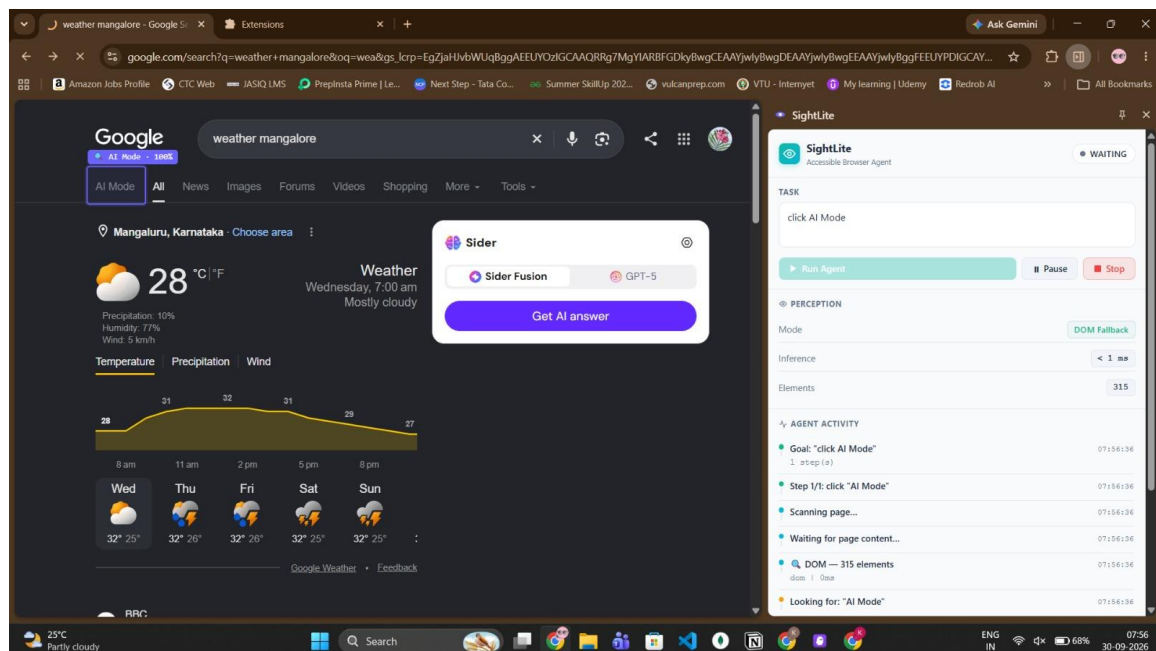


Figure 6.2: SightLite agent executing the goal "click AI Mode" on Google Search

6.3 Discussion

- The agent shows each internal stage in the timeline, which helps users trust and understand what is happening.
- The fallback chain worked as designed: when vision detection was not used, the DOM fallback listed the elements of a real, complex page in very little time.
- Natural language goals removed the need for selectors or scripts.
- Screenshots for perception are processed locally, so the main workflow does not need a cloud vision service.

CHAPTER 7

TESTING

7.1 Test Cases

The following functional test cases were designed to verify the important behaviors of the system.

No.	Test case	Input	Expected result
1	Click a button by label	"click AI Mode"	Element is found, highlighted and clicked
2	Type into a field	"search for cats"	Text "cats" appears in the search box
3	Multi-step goal	"search for cats then scroll down"	Two steps run in order
4	Scroll	"scroll down"	Page scrolls down
5	Explain selection	Select text, right-click, Explain	Short explanation in the timeline
6	Explain page	"explain this article"	Main content is extracted and summarised
7	Destructive action	"click delete account"	Confirmation is requested first
8	Password field	"type 1234 in password"	Action is blocked
9	Vision failure	Model unavailable	DOM scanning is used as fallback
10	Unknown target	"click xyzabc"	Error is reported to the user
11	Stop control	Press Stop during a run	Agent stops immediately
12	Runtime limit	Very long goal	Stops after 15 steps or 90 seconds

CHAPTER 8

LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 Limitations

- Works only in Chromium-based browsers because it uses Manifest V3 APIs.
- Cannot access chrome:// pages, and some sites block extension access.
- Shadow DOM elements are only partly handled, even though vision helps.
- Works on the active tab only; there is no multi-tab support.
- Voice input was removed because Chrome security restrictions block speech recognition inside extension contexts.
- Waits up to five seconds for dynamic content, and stops on critical failures.

8.2 Future Enhancements

- Voice input and output with a proper architecture, including Kannada language support.
- Support for more languages such as Hindi, Tamil and Telugu.
- Cloud synchronisation of sessions across devices.
- Macro recording and replay of workflows, and export to Playwright scripts.
- Firefox support, an offline mode with a local language model, custom model training and a plugin system for custom actions.

CHAPTER 9

CONCLUSION

SightLite demonstrates that a browser assistant can combine natural language understanding, on-device computer vision and robust action execution to make web pages easier to use. The extension turns plain instructions into reliable actions, provides clear real-time feedback, and protects the user through confirmation prompts and safety limits. Its accessibility-oriented design makes it especially useful for visually impaired users, while its automation abilities are also useful to anyone who repeats browser tasks.

The project gave the team practical experience in Chrome extension architecture, ONNX based inference in the browser, prompt-based language understanding and user interface design for accessibility. With voice support, multi-language interfaces and macro recording planned as future work, SightLite can grow into a complete assistive browsing companion.

REFERENCES

- [1] Google Chrome Developers, "Chrome Extensions Manifest V3 documentation", developer.chrome.com/docs/extensions.
- [2] Microsoft, "ONNX Runtime Web documentation", onnxruntime.ai/docs.
- [3] Google, "Gemini API documentation", ai.google.dev.
- [4] S. Ramirez, "FastAPI documentation", fastapi.tiangolo.com.
- [5] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1", www.w3.org/TR/WCAG21.
- [6] W3C, "WAI-ARIA Authoring Practices", www.w3.org/WAI/ARIA/apg.
- [7] MDN Web Docs, "Shadow DOM and DOM events", developer.mozilla.org.
- [8] The PostgreSQL Global Development Group, "PostgreSQL documentation", www.postgresql.org/docs.